

Persona Onboarding: Take-Home Project Plan

48-hour build plan for an adaptive voice + text onboarding agent. Prepared September 23, 2026.

1. The goal

Onboarding is not data collection. It is the first five minutes of using Persona. Every question should either be necessary or double as proof that Persona is useful. By the end, the user should feel like their Persona already gets them, not like they filled out a form.

Guiding principle: depth over breadth. All effort goes into how well the onboarding conversation holds up, not into building extra product surface around it.

2. Confirmed scope and assumptions

Confirmed with the recruiter

- Build the **entire** onboarding flow at full depth, not a stripped-down proof of concept.
- Going deeper with the connected email (for example, bringing up something on the user's calendar) is welcome.
- The yourpersona.com/band page is a marketing and pre-order page. It has no public onboarding flow to copy, so the written spec and the brand's tone are the reference.

Required by the spec

- Collect four things: **agent name**, **user name**, a **connected Gmail**, and **something the user needs help with**.
- Attempt a phone call (a web-simulated voice call is fine) for everything except the agent name.
- Adapt to text as well, and survive user error such as hangups. They will stress test it.
- Feel like a conversation, not a form. Let users "graduate" early. Steer gently when information is needed.

Decisions and assumptions (state these in the README)

- **Gmail is really integrated.** The user signs in with Google (OAuth) through an on-screen "Connect Gmail" card. The agent then reads the user's real upcoming calendar events and recent email subject lines to create the value moment, for example "I see you have a dentist appointment Thursday."
- **Read-only, minimal access.** Persona never sends email or changes the calendar during onboarding. Tokens are stored only on the server, email contents are never logged, and the user can disconnect at any time.
- **The Google app stays in "Testing" mode.** Getting Google to verify an app that reads Gmail takes weeks, so only accounts added as test users can connect. Ask the recruiter which Google accounts the testers will use.
- **A demo-data fallback exists** behind a toggle, only for reviewers who can't sign in with a test account. It is clearly labeled and off by default.
- **A "callback" after a hangup is simulated** in the browser. There are no real phone numbers.

Brand tone to match (from the marketing site): short, confident, casual. "Talk to it the way you'd talk to your friend." "Nothing happens without your yes." The agent should sound like the friendliest, most competent person you know: warm, quick, never salesy.

3. The onboarding journey

This is the path a cooperative user takes. Section 7 covers what happens when users go off it.

Step	Channel	What happens	Why
1. Name your Persona	Text	"First, what do you want to call me?" Suggest 3 names and allow skipping (a default name is used).	The spec says the agent name is never asked by voice. It is also a low-effort, fun first win.
2. Ask to call	Text	"Mind if [Name] gives you a quick 2-minute call? Or we can keep texting."	Consent plus a text option up front. Frustrated or call-averse users never feel trapped.
3. The call	Voice	The agent introduces itself by the name the user just chose, then has a short discovery chat: user's name and what they need help with.	The user hears the name they picked right away, which personalizes it immediately.
4. Connect Gmail	Voice + screen	"I just popped a Connect Gmail button on your screen." The user signs in with Google in a popup while the call keeps going. The agent confirms when it's connected.	Spelling an email out loud is miserable, and speech-to-text gets it wrong. Google sign-in on screen avoids that and gives real access.
5. Value moment	Voice	The agent looks at the user's real upcoming events and recent email subjects, then offers one specific, useful suggestion tied to what they need help with.	This is the moment the user thinks "it's already working," and it uses their real life, not a demo.
6. Graduate	Voice	"Want to jump in? I'll pick up anything else as we go." Always an offer, never automatic.	Respects users who are ready to start. Required by the spec.
7. Recap	Text	After the call, a short text: what I got, what I'll do first, and anything still missing.	Makes progress visible and lets the user fix mistakes, such as a misheard name.

4. How the system thinks

4.1 One brain, two channels

Yes: voice and text both end up as text going into **the same model with the same prompt and the same state**. For voice, speech-to-text turns the user's audio into text, the model replies in text, and text-to-speech speaks the reply. For text, the typed message goes straight in. Only the input and output layers differ, so the agent behaves the same everywhere, and a hangup can seamlessly continue over text.

The catch: turning speech into text throws away tone of voice. A sigh, a sharp tone, or a long hesitation all disappear. Speech-to-text also mangles names and email addresses. The fixes:

- Feed audio signals in as annotations. The voice layer adds tags such as [user interrupted agent], [4.2s silence], or [low transcription confidence: name] to the transcript so the model can react to them.
- Confirm names lightly ("Maya, like M-A-Y-A?") only when transcription confidence is low, and show the name in the post-call recap.
- Never capture the email by voice (see step 4 of the journey).

4.2 Who decides what: code vs. model

The most important design decision: **the model handles language, code handles control flow**. A pure-LLM agent drifts, forgets to ask for things, and is hard to test. A pure state machine feels like a form. The hybrid gets both reliability and warmth.

Code (deterministic orchestrator) owns	The model (LLM) owns
The state: which fields are known or missing	Understanding messy, out-of-order, multi-part answers
Call status, hangups, silence timers, channel switches	Detecting frustration, rambling, corrections, and skip requests
Whether graduation is allowed (a fixed rule)	Wording: warm, varied, never repeating the same line
Validating inputs (email format, name sanity checks)	Choosing the value-moment suggestion based on the user's need
A "director's note" each turn telling the model what to focus on	Deciding how to follow that note naturally
Google sign-in, tokens, and fetching calendar and inbox data	Picking the one item from that data worth mentioning

4.3 What happens on every turn

```

on user_turn(input, channel):
    text = input if channel == "text" else speech_to_text(input) + audio_annotations

    note = orchestrator.directors_note(state)
    # e.g. "Known: user_name=Maya. Missing: gmail, help_topic.
    #       Signals: short answers for 2 turns (maybe rushed).
    #       Graduation: not yet allowed. Priority: learn what she needs help with."

    reply, tool_calls = llm(system_prompt + note, conversation_history + text)

    for call in tool_calls:
        # e.g. save_onboarding_info(user_name="Maya")
        orchestrator.validate_and_apply(call) # code checks it, then updates state

    orchestrator.update_signals(state) # graduation check, stall counter, etc.
    send(reply, channel) # text bubble, or text-to-speech audio

```

4.4 The tool the model calls to save information

Instead of the model outputting a big JSON blob (which delays speech), the model talks normally and calls a tool whenever it learns something. Tool calls are a native feature of current LLMs and voice frameworks.

```

save_onboarding_info(
  user_name?:      string,
  help_topic?:     string,    # user's own words, e.g. "drowning in school emails"
  agent_name?:     string,    # text channel only
  sentiment?:      "neutral" | "frustrated" | "enthusiastic" | "rushed" | "chatty",
  wants_to_skip?:  boolean,
  wants_text?:     boolean,   # user asked to move off the call
  correction_of?:  "user_name" | "help_topic" | "agent_name"
)

```

4.5 The state

```

OnboardingState {
  agent_name, user_name, gmail, help_topic : string | null
  gmail_status      : "not_connected" | "popup_open" | "connected" | "denied" | "error"
  google_tokens     : stored server-side only, never sent to the model or browser
  account_snapshot  : next ~10 calendar events + ~20 recent email subjects (fetched once)
  channel           : "text" | "voice"
  call_status       : "not_started" | "ringing" | "in_progress"
                   | "hung_up" | "declined" | "completed"
  sentiment         : "neutral" | "frustrated" | "enthusiastic" | "rushed" | "chatty"
  turns_since_progress : number    # 3+ means we're stalling: nudge or offer an exit
  graduation_offered, graduated : boolean
  transcript         : Turn[]      # shared by voice and text, survives a hangup
}

```

4.6 The graduation rule

Offer graduation once **help_topic is known AND at least one of user_name or gmail is known**. The help topic is what unlocks being useful, and everything else can be picked up later. It is always phrased as an offer. If declined, don't offer again for at least 3 turns. If the user asks to skip on their own, let them go immediately.

5. Models and technologies

5.1 Recommended stack

Role	Recommended	Why	Alternative
Voice orchestration	LiveKit Agents or Pipecat (open-source voice agent frameworks)	Built-in voice activity detection, turn detection, and interruption (barge-in) handling. Saves days of audio plumbing.	Hosted platforms like Vapi or Retell. Faster, but it can look like you outsourced the hard part.
Speech-to-text	Deepgram streaming (latest Nova model)	Low-latency streaming, word timestamps, confidence scores (used for the annotations in 4.1).	OpenAI transcription models
The brain (LLM)	Claude Sonnet 5	Strong at natural conversation, instruction-following, and tool calls. Streams tokens so speech can start early.	Claude Haiku 4.5 if voice latency is too high, or GPT-class models
Text-to-speech	ElevenLabs (low-latency Flash model) or Cartesia Sonic	Natural, expressive voices with streaming playback. Voice quality was specifically called out.	OpenAI TTS
Gmail and calendar	Google OAuth plus the Gmail and Google Calendar APIs, read-only scopes	Real data for the value moment. Request the smallest scopes possible: sign-in, read calendar, read email.	Nylas (one API for many email providers), but that adds another vendor
Frontend	Next.js or React, with a chat view and a call screen	One page with a text thread, a "call" overlay, and the Connect Gmail card.	Whatever you're fastest in
Testing (evals)	LLM-simulated users plus an LLM judge (Claude)	Automatically runs dozens of difficult conversations. See section 9.	Manual testing only

Before committing: this space moves fast. Check the current model names, pricing, and free tiers for each provider on day one, and pick based on what you can get running in the first two hours.

5.2 The other option: speech-to-speech models

Native speech-to-speech models, such as the OpenAI Realtime API or Gemini Live, skip transcription entirely: the model hears audio and speaks back directly. This is the most "cutting edge" option. Here is the tradeoff:

	Cascaded (recommended)	Speech-to-speech
Hears tone of voice	No, approximated with annotations	Yes, natively
Latency	Good with streaming at every stage	Best
Same brain for voice and text	Yes, identical logic	No, a separate voice model (can share tools and prompt)
Testable and debuggable	Easy: every turn is readable text	Harder: audio in, audio out
Voice choice and quality	Any TTS voice you like	Limited to the provider's voices
Risk in 48 hours	Low	Medium to high

Recommendation: build cascaded, because one shared brain is what makes the hangup-to-text handoff and testing reliable. In the README, explain that you evaluated speech-to-speech and why you chose cascaded. Showing that you understand the tradeoff signals as much expertise as using the newest API. If you have spare time, a speech-to-speech mode behind a toggle is a strong stretch goal.

5.3 What makes this feel cutting-edge

- Real interruption handling: the user can talk over the agent and it stops and listens, like a person would.
- Mid-call handoff to the screen (the Gmail card) while the voice conversation keeps going. Multimodal, not just voice.
- A seamless switch from call to text with full context after a hangup.
- A personalized value moment using the user's real calendar and inbox, during onboarding itself.
- An automated eval harness with simulated adversarial users. Few candidates will have this, and it directly answers "I will be stress testing this."

6. Design principles

Principle	What it looks like in practice
Extract, don't interrogate	Pull every field out of whatever the user says. If they answer three things at once, never re-ask any of them.
Reflect before redirecting	Say back what you heard ("Ugh, school emails, got it") before moving on. This is what makes steering feel gentle.
Show value mid-flow	When the user says what they need, respond with one specific idea for it, not "Great, I can help with that!"
Match their energy	Terse user: go fast, skip small talk. Chatty user: let them talk, then bridge back. Frustrated user: shortest path, offer an exit.
Progress is felt, not counted	No "step 2 of 4." Use phrases like "one last thing and I can get started," plus the post-call recap.
Exit is always one sentence away	Skip, pause, switch to text, or jump into the product are available at any point.
Only ask for what's needed	Never pressure. If the user doesn't want to connect Gmail now, respect it and move on.

7. Edge cases

Build these directly. Each one needs intentional behavior, not just "doesn't crash."

Situation	What the agent does
Hangs up mid-call	Save state. Text follows right away: "Looks like we got cut off! Want me to call back, or finish up here?" Continue from exactly where it stopped.
Declines the call	No pushback. Run the full onboarding over text with the same brain.
Doesn't answer or the call rings out	Text: "Missed you! No worries, we can do it here." Offer a callback button.

Situation	What the agent does
Frustrated or rude	Drop small talk, apologize briefly, and ask for only the single most important missing piece. Offer text or skip.
Rambles or goes off-topic	Let them finish. React to something specific they said, then bridge back in one line.
Asks an unrelated question	Answer briefly if possible (a free value moment), then bridge back.
Multiple answers at once	Save everything. Only ask for what's still missing.
Corrects earlier info	Overwrite it, confirm lightly ("Sam, got it"), and move on. No fuss.
Silence on the call	After about 5 seconds: one gentle check-in. After about 10 more: offer to switch to text. Never repeat the same line.
Talks over the agent	Stop speaking immediately and listen (barge-in).
Gives a fake or joke name	Play along with humor, and keep it unless they correct it.
Closes the Google popup or denies access	No guilt: "No problem, you can connect it later." Keep going, and offer it once more at the end.
Google shows an "unverified app" warning	Warn them ahead of time: "Google will show a warning since this is a test app. Tap Advanced, then continue."
Account isn't a test user	Explain it plainly, and offer the demo-data mode or skipping for now.
Empty calendar or new account	Don't fake it. Base the value moment on what they said they need instead.
Google token expires or the API fails	Retry once, then continue without the data. Never read an error message out loud.
Private items in the inbox	Only mention calendar events and email subjects, never email bodies. Skip anything that looks medical, financial, or personal.
Wants to skip everything	Let them graduate immediately with sensible defaults. Missing fields get picked up naturally later.
Tries to derail or jailbreak the agent	Stay friendly and in character, then steer back. The system prompt defines scope.
Refreshes the page or leaves and comes back	Resume the session from saved state: "Welcome back! We were just talking about..."

8. Build plan, step by step

Rough hours are coding time. The schedule assumes you sleep, which you should. Build the brain in text first, and prove the voice pipeline works early so there are no surprises later.

Phase 0: Setup and de-risking (about 2 hours)

- 1 Create the repo with a backend and frontend. Get API keys for the LLM, speech-to-text, and text-to-speech providers.
- 2 Start the Google Cloud setup (see Phase 3) and message the recruiter for the testers' Google accounts, so that answer arrives while you build.

- 3 **Voice spike:** get a "hello world" voice call working end to end in the chosen framework (speak, hear a reply). If this fights you for more than 2 hours, fall back to push-to-talk turn-based voice.
- 4 Set up the state object and a session store keyed by session ID (in-memory or SQLite is fine).

Phase 1: The brain, text only (about 7 hours, the most important phase)

- 1 Write the system prompt: persona and tone, the four fields, extract-first behavior, the value-moment instruction, and scope rules.
- 2 Implement the `save_onboarding_info` tool and its validation in code.
- 3 Implement the orchestrator: the director's note, stall counter, and graduation rule.
- 4 Build the chat UI: agent name step (step 1 of the journey) and the ask-to-call step (step 2).
- 5 Handle corrections, skip requests, frustration, and rambling. Test each by typing.
- 6 Save transcripts per session so any conversation can be reviewed later.

Phase 2: Voice (about 6 hours)

- 1 Connect the voice pipeline (speech-to-text, then the same brain, then text-to-speech) to the orchestrator.
- 2 Add audio annotations: interruptions, silence lengths, and low-confidence words.
- 3 Build the call screen: ringing, in-call, and an "End call" button that triggers hangup handling.
- 4 Silence timers: check in, then offer to switch to text.
- 5 Hangup to text handoff and the callback button. Test mid-sentence hangups repeatedly.

Phase 3: Real Gmail integration and the value moment (about 6 hours)

- 1 **Do the Google setup on day one**, during Phase 0, since it can hit surprises: create a Google Cloud project, turn on the Gmail and Calendar APIs, set up the consent screen in Testing mode, and add yourself and the testers as test users.
- 2 Build the sign-in flow: the Connect Gmail button opens a Google popup, the server receives the tokens and stores them, and the orchestrator marks Gmail as connected. The agent confirms by voice while the call is still going.
- 3 Right after connecting, fetch a small snapshot: the next ~10 calendar events and ~20 recent email subject lines. Keep this short and summarized so it doesn't slow down the model.
- 4 Value moment: once Gmail is connected and the help topic is known, give the model the snapshot and have it pick one relevant item and offer one concrete action ("Want me to draft a reply to that?"). Offer only; don't act.
- 5 Handle every failure case in section 7: popup closed, access denied, not a test user, empty account, API errors.
- 6 Add a Disconnect button and delete tokens when the session ends.
- 7 Build the demo-data fallback toggle last, only if time allows.

Phase 4: Graduation and recap (about 1.5 hours)

- 1 The graduation offer and a simple "You're in" landing screen: the agent's name, what it knows, and its first suggested task.
- 2 Post-call text recap with edit links for each field.

Phase 5: Stress testing (about 4 hours, non-negotiable)

- 1 Run every edge case in section 7 by hand, on both voice and text.
- 2 Build the automated eval harness described in section 9 and run it. Fix what breaks.

Phase 6: Polish and submission (about 3 hours)

- 1 Vary the agent's phrasing so repeated runs don't sound scripted. Tune voice speed and pauses.
- 2 Write the README (see section 10).
- 3 Record a 2-3 minute demo video: a happy path, a hangup recovery, and an angry-user run.
- 4 Deploy it (for example, Vercel plus a small backend host) so reviewers can try it without setup. Clone fresh and follow your own README once.

Suggested schedule

Window	Work
Hours 0-12	Phase 0 (including Google Cloud setup) and Phase 1. By bedtime, the text-only brain handles the full flow.
Hours 12-20	Sleep.
Hours 20-36	Phases 2, 3, and 4. Voice, real Gmail integration, value moment, graduation.
Hours 36-42	Phase 5 and Phase 6. Stress testing, polish, README, video, deploy.
Hours 42-44	A fresh-eyes run-through as a first-time user, then submit. Hours 44-48 are emergency buffer.

9. Testing: simulated difficult users

Because text and voice share one brain, most testing can happen in text, fast and automated:

- **Simulated users:** a script where an LLM plays a user persona and chats with your agent. Personas: the rushed exec, the rambler, the skeptic, the joker with a fake name, the person who answers everything in one message, the one who corrects themselves, the one who hangs up after two turns, the one who refuses Gmail, the one who tries to derail the agent.
- **An LLM judge** scores each finished conversation: were all fields captured correctly? Was anything asked twice? Was the tone warm? Was graduation offered at the right time? Did it ever feel like a form?
- **Report** a simple table of pass rates in the README. "Ran 50 adversarial conversations, 47 passed, here's what I fixed" is a very strong signal.

10. Estimated costs

These are estimates for planning. Claude prices come from Anthropic's current price list. Prices for the other vendors change often, so check each pricing page on day one and set a spending limit in every dashboard.

10.1 Cost of one full onboarding conversation

Assumes about 20 back-and-forth turns, a ~3,000-token system prompt, and the conversation history resent each turn (about 6,000 input tokens per turn on average, and about 150 output tokens).

Piece	Usage per conversation	Approximate cost
Claude Sonnet 5 (the brain), \$2 in / \$10 out per million tokens	~120,000 input tokens, ~3,000 output tokens	~\$0.27 without caching, ~\$0.08-0.12 with prompt caching
Speech-to-text (Deepgram streaming)	~5 minutes of call audio	~\$0.04
Text-to-speech	~3,000 characters spoken by the agent	~\$0.02-0.10, depending on provider and plan
Google Gmail and Calendar APIs	A few read requests	Free
Total for one voice onboarding		~\$0.15-0.40

10.2 The 48-hour build, with and without automated testing

Item	Build only (manual testing)	Build + automated evals	Notes
Claude API: development	\$15-50	\$15-50	About 150-300 practice conversations while building and debugging.
Claude API: eval harness	-	\$30-120	Each simulated conversation costs ~\$0.20-0.45: your agent, the simulated user (Haiku 4.5), and the judge (Opus 5, \$5/\$25). 50 conversations per round, 3-6 rounds.
Speech-to-text	\$0-5	\$0-5	Several hundred minutes of test calls. Deepgram usually gives new accounts free credit that covers this.
Text-to-speech	\$5-25	\$5-25	A starter plan (Cartesia around \$5/month, ElevenLabs around \$22/month). Evals run in text, so they add nothing here.
Voice framework	\$0	\$0	Pipecat is open source. LiveKit Cloud has a free tier.
Hosting	\$0-10	\$0-10	Frontend on a free tier (for example, Vercel). A small always-on backend for the voice agent (for example, Railway or Render).
Google Cloud (OAuth, Gmail, Calendar)	\$0	\$0	Free at this scale.
Total	~\$20-90	~\$50-210	

After you submit: keep about \$10-25 of headroom for the team's stress testing, since they may run 20-50 conversations, some by voice. Keep the services running until you hear back.

10.3 How to keep costs at the low end

- **Use prompt caching.** The system prompt and tool definitions are the same on every turn, so caching them cuts the Claude bill by roughly half to two-thirds. It's the single biggest saving.
- **Run evals in text only.** Since voice and text share one brain, automated tests don't need speech-to-text or text-to-speech at all.
- **Use cheaper models where quality matters less.** Claude Haiku 4.5 is fine for the simulated user. Keep the stronger models for your agent and the judge.
- **Grade evals with the Batch API**, which costs 50% less. Grading doesn't need to be instant.
- **Keep the account snapshot short** (event titles and times, email subject lines) so every turn doesn't resend a lot of text.
- **Set hard spending limits** in the Anthropic, Deepgram, and text-to-speech dashboards before you start.

Rule of thumb: plan on about **\$100** for the whole project, including automated testing and the team's review. Build-only with careful caching can come in under \$40.

11. README checklist and definition of done

The README should cover

- What it is, a link to the live demo, a link to the video, and how to run it locally in under 5 minutes.
- The architecture diagram and the "model handles language, code handles control flow" reasoning.
- The cascaded vs. speech-to-speech tradeoff and why you chose cascaded.
- How the Gmail integration works: which scopes and why, what's read, where tokens live, and how to get added as a test user.
- What's simulated (the phone call and callbacks) and why.
- The edge-case table and the eval results.
- What you'd do with more time (below).

Done means

- All four fields can be collected over voice, text, or a mix of both.
- A test user can connect a real Gmail account mid-call, and the agent mentions something real from it.
- Every edge case in section 7 has intentional, visible behavior.
- Graduation follows the rule and is never forced.
- The eval harness runs and its results are in the README.
- A stranger can open the deployed link and complete onboarding without instructions.

What you'd do with more time (for the README)

- Complete Google's app verification so anyone can connect, not just test users.

- Real telephony (for example, Twilio) so the agent can actually call a phone number.
- A speech-to-speech mode that hears tone of voice directly.
- Learning from drop-off analytics: which step loses people, and fixing that first.